

Tools of the Trade

EART22001 / EART60071

This section is a practical guide to the computing environment used in the modelling practicals for EART22001 and EART60071. You should be able to work through it from the beginning on either Windows or macOS, even if you have not used a command line before.

For most practicals, the numerical model will run on a **Linux server**. Your laptop or desktop is the computer you sit in front of; it is used to connect to the server, transfer files and inspect results. Once you connect with SSH, the commands you type are being run on the remote Linux machine.

1 What you need before you start

Have the following ready:

- a Windows or macOS computer with an internet connection;
- the on-campus server address, 130.88.66.57;
- the **course server username and password** shown on Canvas;
- a terminal application: PowerShell, Command Prompt or Windows Terminal on Windows, or Terminal on macOS.

The server login is not your central University login. Use the course-specific username and password supplied on Canvas. Do not put passwords into scripts, screenshots, Git repositories or discussion posts.

For on-campus practicals, the server address is 130.88.66.57. Canvas remains the authoritative source for the username/password and for any announcement that the address has changed.

In the examples below, `YOUR_USERNAME` means the course username supplied on Canvas and `SERVER_ADDRESS` means 130.88.66.57 when you are working on campus. When a terminal example shows a prompt, the prompt is only a visual cue: type the command that follows it, not the prompt itself.

Commands, filenames, paths and literal messages are shown in a shaded monospace style. Larger shaded boxes contain complete commands or terminal sessions: type only the command text, not the surrounding box.

2 The basic workflow

Most practicals follow the same pattern:

1. open a terminal on your own computer;
2. connect to the course server using **SSH**;
3. move to the directory for the practical and obtain the code or data;
4. edit an input or configuration file if required;
5. run the model on the server;
6. inspect the output;
7. transfer any results you want to keep back to your own computer;
8. analyse or plot the results, usually with Python.

A useful distinction throughout this guide is:

Local computer = your Windows PC or Mac.

Remote computer = the Linux modelling server.

SSH gives you a terminal *on the remote computer*. Commands such as `scp` that copy files to or from your laptop are normally started from a *local* terminal.

3 Windows: connect with PowerShell, Command Prompt or Windows Terminal

Windows Terminal is an application that can host several command-line shells. For this course, **PowerShell** is a good default. If PowerShell is not available or is blocked on a managed computer, **Command Prompt**

(**cmd**) works for the SSH, SFTP and SCP commands used in this course.

1. Open a terminal. Press the Windows key and open **PowerShell** or **Windows Terminal**. If these are unavailable, search for **Command Prompt** or **cmd** instead.

2. Check that SSH is available. Type:

```
ssh -V
```

If you see an OpenSSH version, you are ready. If Windows reports that `ssh` is not recognised, the OpenSSH Client may need to be enabled in Windows Optional Features. On a University-managed computer, do not change system settings without permission; ask a graduate teaching assistant (GTA) if the client is not available. See the [Microsoft OpenSSH documentation](#) if you need Windows-specific installation help.

3. Connect to the course server.

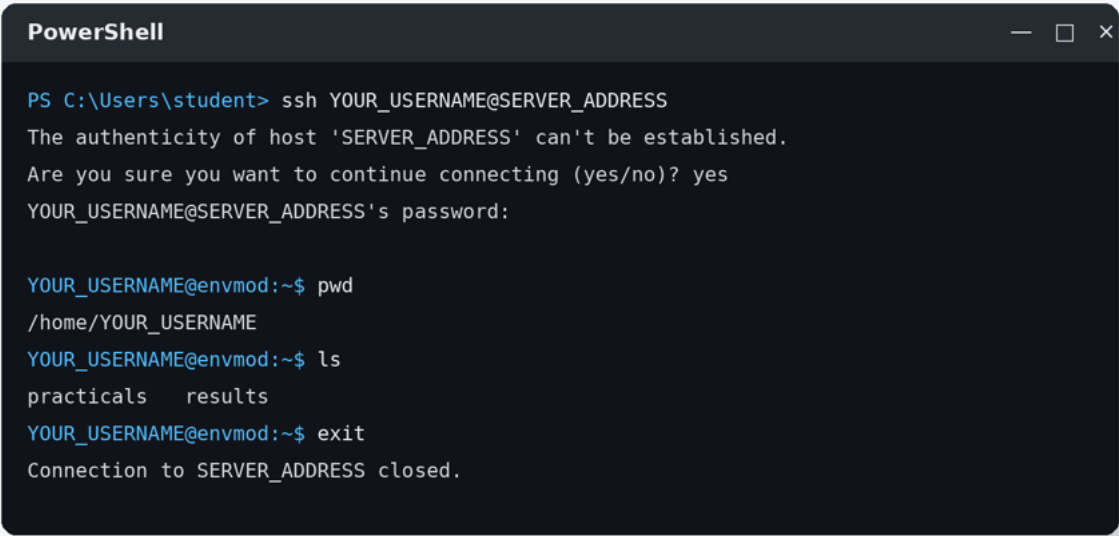
```
ssh YOUR_USERNAME@SERVER_ADDRESS
```

For example, on campus this is:

```
ssh YOUR_USERNAME@130.88.66.57
```

The first time you connect to a server, SSH may ask whether you trust its host key. Check that the server address exactly matches Canvas before answering yes. You will then be asked for the course password.

Nothing appears while you type an SSH password. There are no dots or asterisks. This is deliberate; type the password and press Enter.



```
PowerShell
PS C:\Users\student> ssh YOUR_USERNAME@SERVER_ADDRESS
The authenticity of host 'SERVER_ADDRESS' can't be established.
Are you sure you want to continue connecting (yes/no)? yes
YOUR_USERNAME@SERVER_ADDRESS's password:

YOUR_USERNAME@envmod:~$ pwd
/home/YOUR_USERNAME
YOUR_USERNAME@envmod:~$ ls
practicals  results
YOUR_USERNAME@envmod:~$ exit
Connection to SERVER_ADDRESS closed.
```

Illustrative PowerShell session. Your prompt, username, server name and first-connection message will differ.

Once the remote prompt appears, you are logged in to Linux. Try:

```
pwd
ls
```

To finish the remote session cleanly, type `exit`.

4 macOS: connect with Terminal

macOS includes both Terminal and the SSH command-line tools.

1. Open Terminal. Press Command-Space to open Spotlight, type **Terminal**, and press Return. You can also find it in /Applications/Utilities.

2. Check SSH.

```
ssh -V
```

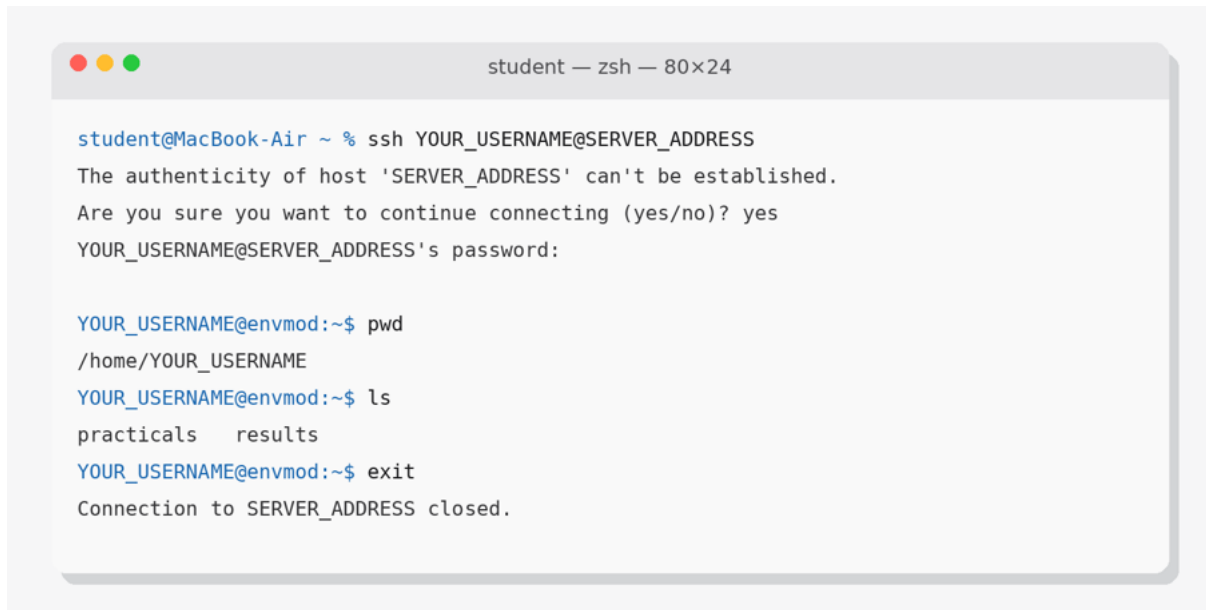
3. Connect.

```
ssh YOUR_USERNAME@SERVER_ADDRESS
```

On campus, use:

```
ssh YOUR_USERNAME@130.88.66.57
```

As on Windows, check the server name carefully if you are shown a first-connection host-key question, then enter the course password. The password will not be displayed while you type it. Apple provides a concise [Terminal User Guide](#) if you want more detail.



Illustrative macOS Terminal session. The default macOS shell is commonly zsh, while the remote server prompt shown after login is Linux.

Type exit when you want to disconnect from the server.

5 How to tell whether you are local or remote

This is one of the most important habits to develop. Look at the prompt before running a command.

Typical prompts look like this:

Where you are	Example prompt
Windows (local)	PS C:\Users\student>
macOS (local)	student@MacBook-Air ~ %
Linux server (remote)	YOUR_USERNAME@envmod:~\$

If you are unsure where you are, type `hostname` and `pwd`. The former tells you which computer is running the shell; the latter tells you which directory you are in.

6 Essential Linux commands on the server

You do not need a large Linux vocabulary. The commands below cover most routine work in either module.

Important: words in CAPITALS below are placeholders, not words to type literally. For example, `cd DIR` means “type `cd` followed by the name of the directory you actually want”, such as `cd practical1`. Likewise, `mv OLDNAME NEWNAME` could become `mv output.dat output_old.dat`.

Command pattern	What it does
<code>pwd</code>	print the full path of the current directory
<code>ls</code>	list files and directories
<code>ls -lh</code>	list files with readable sizes and more detail
<code>cd DIR</code>	enter a directory, for example <code>cd practical1</code>
<code>cd ..</code>	move up one directory level
<code>cd ~</code>	return to your home directory
<code>mkdir NAME</code>	create a directory, for example <code>mkdir results</code>
<code>nano -l FILE</code>	edit a text file in nano and display line numbers
<code>less FILE</code>	read a text file one screen at a time; press <code>q</code> to quit
<code>cp SOURCE TARGET</code>	copy a file, for example <code>cp input.txt input_backup.txt</code>
<code>mv OLDNAME NEWNAME</code>	move or rename a file, for example <code>mv run1.dat run1_old.dat</code>
<code>rm -i FILE</code>	remove a file, asking you to confirm before deletion
<code>hostname</code>	show the name of the computer you are logged into
<code>exit</code>	leave the remote SSH session

Be careful with `rm`. On the Linux server, deleted files do not normally go to a recycle bin. For beginners, `rm -i FILE` is preferable because it asks for confirmation. Plain `rm FILE` removes the file immediately. Before deleting anything important, use `pwd` and `ls` to make sure you are in the directory you think you are in and that the filename is correct.

Three keyboard habits save a lot of time:

- **Tab completion:** type the first few letters of a file or directory and press `Tab`. The shell will complete it when possible.
- **Up arrow:** recalls previous commands, which is safer than retyping a long command.
- **Ctrl+C:** asks a running command to stop. Use this when you started the wrong command or a program is waiting indefinitely.

Linux file names are case-sensitive: `Results.csv` and `results.csv` are different files. Avoid spaces in working-directory and file names unless you are comfortable quoting them.

7 Editing a file with nano

For small configuration files, `nano` is sufficient and runs entirely in the SSH terminal. I recommend using the `-l` option so that line numbers are displayed down the left-hand side:

```
nano -l input.txt
```

This is particularly useful when a practical tells you to change a particular line or when you are comparing your file with a worked example. Edit the text normally. At the bottom of `nano`, the symbol `^` means the `Ctrl` key. The most useful shortcuts are:

- **Ctrl+O**, then `Enter`: save (“write out”) the file;

- **Ctrl+X**: exit nano;
- **Ctrl+W**: search within the file.

8 Moving files between your computer and the server

There are two straightforward command-line approaches: `scp` for quick copies and `sftp` for an interactive file-transfer session. Both use SSH and the same course credentials.

Quick copies with scp

Run these commands in a **local** PowerShell/Command Prompt/Terminal window, not inside the SSH session.

Copy a local file to your server home directory:

```
scp myfile.py YOUR_USERNAME@SERVER_ADDRESS:~/
```

Copy a file from the server into the current local directory:

```
scp YOUR_USERNAME@SERVER_ADDRESS:~/results.csv .
```

Copy a whole directory from the server:

```
scp -r YOUR_USERNAME@SERVER_ADDRESS:~/results .
```

Here `~/` means your home directory on the server and the final dot means “the current directory on my local computer”. If you are unsure where that is, run `pwd` in your *local* terminal before the `scp` command. After the transfer, `explorer .` on Windows or `open .` on macOS opens that folder graphically.

Browsing with sftp

Start SFTP from a local terminal:

```
sftp YOUR_USERNAME@SERVER_ADDRESS
```

Once connected, remember that commands without an `l` refer to the **remote** server; commands beginning with `l` refer to the **local** computer.

SFTP command	Meaning
<code>pwd / ls</code>	show the remote directory / list remote files
<code>cd DIR</code>	change remote directory
<code>lpwd / ll</code>	show the local directory / list local files
<code>lcd ~</code>	change the local directory to your home folder
<code>lcd DIR</code>	change the local directory to another folder
<code>get FILE</code>	download a remote file into the current local directory
<code>put FILE</code>	upload a local file from the current local directory
<code>exit</code>	close SFTP

A useful way to think about SFTP is that it keeps track of **two directories at the same time**:

- `pwd` tells you the remote directory on the Linux server;
- `lpwd` tells you the local directory on your Windows PC or Mac;
- `get FILE` copies from the remote directory shown by `pwd` to the local directory shown by `lpwd`;
- `put FILE` copies in the opposite direction.

Windows note. On some Windows computers, `lpwd` may initially show a protected folder such as `C:\Windows\System32`; in that case `get` may fail. At the `sftp>` prompt, type `lcd ~` to move to your local home folder, then `lcd Downloads` if you want to save into Downloads. Type `lpwd` again before using `get`.

Where did my downloaded file go?

This is one of the most common sources of confusion with SFTP. A file downloaded with `get` is placed in SFTP's *current local directory*. Before downloading, type

```
lpwd
```

SFTP will print the local folder it is using. For example, on Windows it might report something like `C:/Users/student/Downloads`; on a Mac it might report `/Users/student/Downloads`. If you then type `get results.csv`, the file `results.csv` will appear in that folder on your own computer.

A good habit is to choose the local destination *before* transferring files. For example, if you want downloads to go into your normal Downloads folder:

Windows PowerShell, before starting SFTP:

```
cd $HOME\Downloads
sftp YOUR_USERNAME@130.88.66.57
```

Windows Command Prompt (cmd), if PowerShell is unavailable:

```
cd %USERPROFILE%\Downloads
sftp YOUR_USERNAME@130.88.66.57
```

Then, inside SFTP:

```
lpwd
get results.csv
exit
```

Back in PowerShell or Command Prompt, open the folder in File Explorer with:

```
explorer .
```

macOS (Terminal), before starting SFTP:

```
cd ~/Downloads
sftp YOUR_USERNAME@130.88.66.57
```

Then, inside SFTP:

```
lpwd
get results.csv
exit
```

Back in Terminal, open the folder in Finder with:

```
open .
```

You can also change the local destination while SFTP is running with `lcd DIR`. After doing so, type `lpwd` again to confirm where files will be saved. If you cannot find a transferred file, do not transfer it repeatedly: reconnect, use `lpwd`, and check that folder first.

9 Getting practical code with Git

Git is a version-control system; GitHub is a service that hosts Git repositories. You do not need to learn advanced Git for the first practical. The main command is `git clone`, which creates a local copy of a repository on the server.

After logging into the modelling server with SSH:

```
cd ~
git clone https://github.com/EnvModelling/approximate-methods-practical
cd approximate-methods-practical
ls
```

If you return to a repository later and have been told that the teaching material has changed, enter the repository and use `git pull` before starting work. Do not use `git pull` blindly if you have made substantial changes of your own; Git may ask you to resolve them.

10 A first-session checklist

Before you begin the scientific part of the first practical, make sure you can do all of the following without assistance:

1. find the current server address and course-specific credentials on Canvas;
2. open PowerShell/Command Prompt/Windows Terminal or macOS Terminal;
3. connect with `ssh YOUR_USERNAME@SERVER_ADDRESS`;
4. recognise that the prompt has changed from your local computer to the remote server;
5. run `hostname`, `pwd` and `ls`;
6. create a directory, enter it and return to your home directory;
7. create and save a short file with `nano -l`;
8. open a *second local terminal* and copy that file back with `scp` or `sftp`;
9. log out of SSH with `exit`.

If you can complete that sequence, you have the core computing skills required for the rest of either module.

11 Troubleshooting

Most connection problems are simple. Read the message in the terminal before trying random fixes.

What you see	What to check
ssh is not recognised	On personal Windows computers, check that OpenSSH Client is enabled. On managed computers, ask a GTA rather than changing protected settings.
Could not resolve hostname	Usually a typo in the server address. On campus it should be 130.88.66.57; do not include spaces.
Connection timed out	Check your network connection and that you are using 130.88.66.57 on campus. If several students see the same error, ask a GTA because the server may be unavailable.
Permission denied	Recheck the course username and password. They are not your central University credentials. Password entry is invisible.
REMOTE HOST IDENTIFICATION HAS CHANGED	Stop and ask a GTA or lecturer before bypassing the warning. It can occur after a legitimate server rebuild, but SSH displays it because an unexpected host-key change can also be a security problem.
No such file or directory	Run <code>pwd</code> and <code>ls</code> ; you are probably in the wrong directory or have a spelling/capitalisation mismatch.
A command appears to hang	Give it a moment; if you are sure it should not be running, press Ctrl+C.

When asking for help, it is much more useful to show the **exact command and exact error message** than to say only that “the server does not work”. Never include your password in a screenshot.

12 Docker: optional way to work locally from home

The normal practical-class workflow is to connect to the course modelling server using SSH. Access to this server is restricted to the University campus network. You can therefore use your own laptop to connect to the server while you are on campus, but you cannot normally connect to it directly from home.

If you want to practise the practical exercises away from campus, you can instead run the **EnvModelling Docker environment** on a suitable personal computer. Docker provides a local Linux environment containing the software needed by the course models. It also supports SSH and SFTP, so you can practise essentially the same command-line workflow that you use with the University server.

Docker is optional. You do not need to install it for the normal on-campus practical classes.

Installing EnvModelling

Full installation instructions for Windows, macOS, Linux and supported Chromebooks are maintained in the EnvModelling repository:

github.com/EnvModelling/EnvModelling

Follow the instructions there to install Docker and set up the local environment. The repository contains separate guidance for different operating systems.

Once installed, the usual startup commands, run from the downloaded EnvModelling directory on your own computer, are:

```
docker compose pull
docker compose up -d
```

Then connect to your local modelling environment with:

```
ssh -p 2022 student@localhost
```

The container uses the normal SSH port 22 internally, but Docker exposes it as port 2022 on your computer to avoid host conflicts. SSH therefore needs lower-case -p; SFTP uses upper-case -P: `sftp -P 2022 student@localhost`.

The local Docker password is `envmodelling`. This is for the local teaching container only; it is not your University password or course-server password.

Where to keep your work

After logging in you should start in:

```
/home/student/work
```

which can also be written as `~/work`.

Keep model repositories, model output and other work in this directory. It is mapped to the `student_work` directory in your EnvModelling folder on your own computer and is retained when the Docker container is updated or replaced.

For example, you can clone the repository required by a practical from inside `~/work`:

```
git clone REPOSITORY_URL
```

The individual practical sheets give the appropriate repository address.

University server or Docker?

	University modelling server	Docker
Practical classes	Use this	Optional
On campus	Available	Available on your own computer
At home	Normally unavailable	Available after installation
Connection	SSH/SFTP to course server	SSH/SFTP to localhost
Working environment	Linux	Linux

The scientific models and practical tasks are the important part: Docker is simply another way of providing the Linux environment in which they run.

13 What these tools are for

The command line is only the machinery. The scientific goal is to understand what a model represents and why one model is appropriate for one environmental problem but not another. Across the two modules you will encounter:

- **ODE models**, where the state of a system evolves in time;
- **empirical or semi-empirical models**, which combine theory with parameters constrained by observations;
- **reduced-dimensional models**, which simplify geometry to isolate important processes;
- **fluid-dynamical models**, which solve equations describing motion and transport in fluids such as the atmosphere and ocean.

A question worth asking every time you run a model is: *what has the model retained, what has it neglected, and are those choices appropriate for the problem?*